



URI、URL和URLConnection

北京理工大学计算机学院
金旭亮

URI



- 1 URI: Uniform Resource Identifier
- 2 用于给互联网上的资源起一个“唯一”的名字。
- 3 相关规范:

《Uniform Resource Identifiers (URI): Generic Syntax》

<https://www.ietf.org/rfc/rfc2396.txt>

《Format for Literal IPv6 Addresses in URL's》

<https://www.ietf.org/rfc/rfc2732.txt>

URI示例



```
public class UseURI {  
    public static void main(String[] args) {  
        var myWebsite :URI = URI.create("https://weibo.com:9000/zhanshan/home?wvr=5");  
        System.out.println(myWebsite.getAuthority()); //输出: weibo.com:9000  
        System.out.println(myWebsite.getPath()); //输出: /zhanshan/home  
        System.out.println(myWebsite.getHost()); //输出: weibo.com  
        System.out.println(myWebsite.getPort()); //输出: 9000  
        System.out.println(myWebsite.getScheme()); //输出: https  
        System.out.println(myWebsite.getQuery()); //输出: wvr=5  
    }  
}
```

什么是URL



URL是“**统一资源定位符 (Uniform Resource Locator)**”的简称，它表示Internet上某一资源的地址。通过URL，就可以访问Internet。浏览器或其他程序通过解析给定的URL就可以在网络上查找相应的文件或其他资源。

URL规范，见

<http://www.ietf.org/rfc/rfc2396.txt>

URL编码与解码



```
public class URLDecoderTest {  
    public static void main(String[] args)  
        throws Exception {  
        // 将普通字符串转换成  
        // application/x-www-form-urlencoded字符串  
        String urlStr = URLEncoder.encode(  
            s: "中国人", enc: "UTF-8");  
        System.out.println(urlStr);  
  
        // 将application/x-www-form-urlencoded字符串  
        // 转换成普通字符串  
        String keyWord = URLDecoder.decode(  
            s: "%E4%B8%AD%E5%9B%BD%E4%BA%BA", enc: "utf-8");  
        System.out.println(keyWord);  
    }  
}
```

Run: URLDecoderTest x

▶ ↑ "C:\Program Files\Java\jdk-17\bin\java.exe"
⚙ ↓ %E4%B8%AD%E5%9B%BD%E4%BA%BA
☐ ↺ 中国人
🔄 ⏮ Process finished with exit code 0

Java编程中的URL



- Java中使用URL对象代表一个URL

```
URL url1 = new URL("http://www.example.com/index.html");  
URL url2 = new URL("http", "www.example.com", "/index.html");  
URL url3 = new URL("http", "www.example.com", 80, "/index.html");
```

- 可以基于已有的URL（称为“base URL”）定义新的URL。

```
URL myURL = new URL("http://example.com/pages/");  
URL page1URL = new URL(myURL, "page1.html");  
URL page2URL = new URL(myURL, "page2.html");
```



一个URL对象生成后，其属性是不能被改变的。

非法URL



当创建URL对象时，如果给定的URL字符串不合法，将抛出 `MalformedURLException`。

```
try {  
    URL myURL = new URL(...);  
}  
catch (MalformedURLException e) {  
    // exception handler code here  
    // ...  
}
```

通过URL读取www信息



URL 类提供了一个openStream()方法:

```
public final InputStream openStream();
```

此方法与指定的URL建立连接并返回一个InputStream对象，通过此对象可以读取网络资源中的数据。

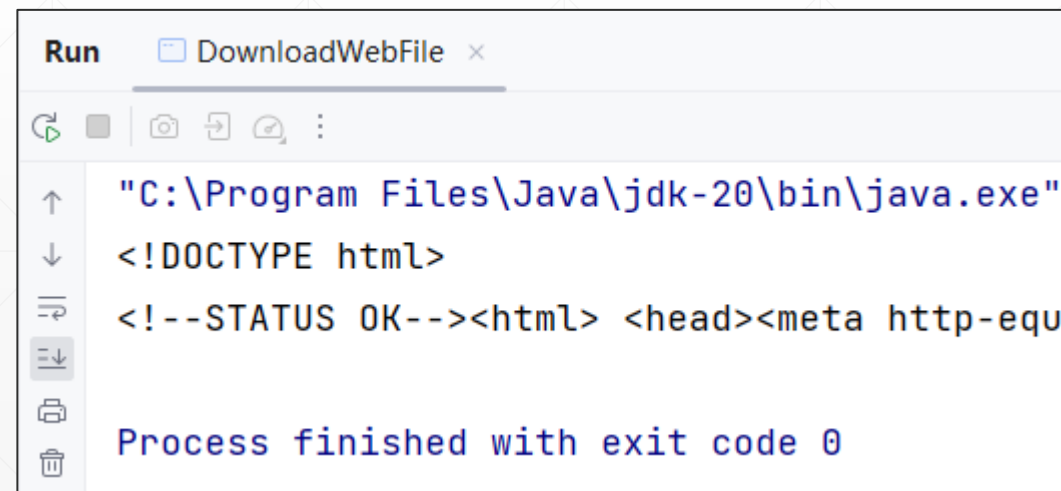


从Web下载文件示例代码

```
public static void main(String[] args) {
    BufferedReader bufferedReader=null;
    try{
        //创建URL对象
        URL url = new URL("http://www.baidu.com/");
        //获取输入流
        InputStream inputStream = url.openStream();
        //从流中读取数据，注意其编码
        bufferedReader = new BufferedReader(
            new InputStreamReader(inputStream,
                Charset.forName("utf-8")));
        String line = bufferedReader.readLine();
        while (line != null) {
            System.out.println(line);
            line = bufferedReader.readLine();
        }
    } catch (MalformedURLException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    }
    finally {
        if(bufferedReader!=null) {
            try {
                bufferedReader.close();
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
}
```

示例：DownloadWebFile.java

运行结果



```
Run  DownloadWebFile x
C:\Program Files\Java\jdk-20\bin\java.exe
<!DOCTYPE html>
<!--STATUS OK--><html> <head><meta http-equ
Process finished with exit code 0
```

URLConnection



URLConnection代表一个到远程计算机的连接，它不仅可以下载数据，也能上传数据。



通过URL对象的openConnection方法获取URLConnection对象。



URLConnection对象的doInput和doOutput字段用于设置是下载还是上传数据。



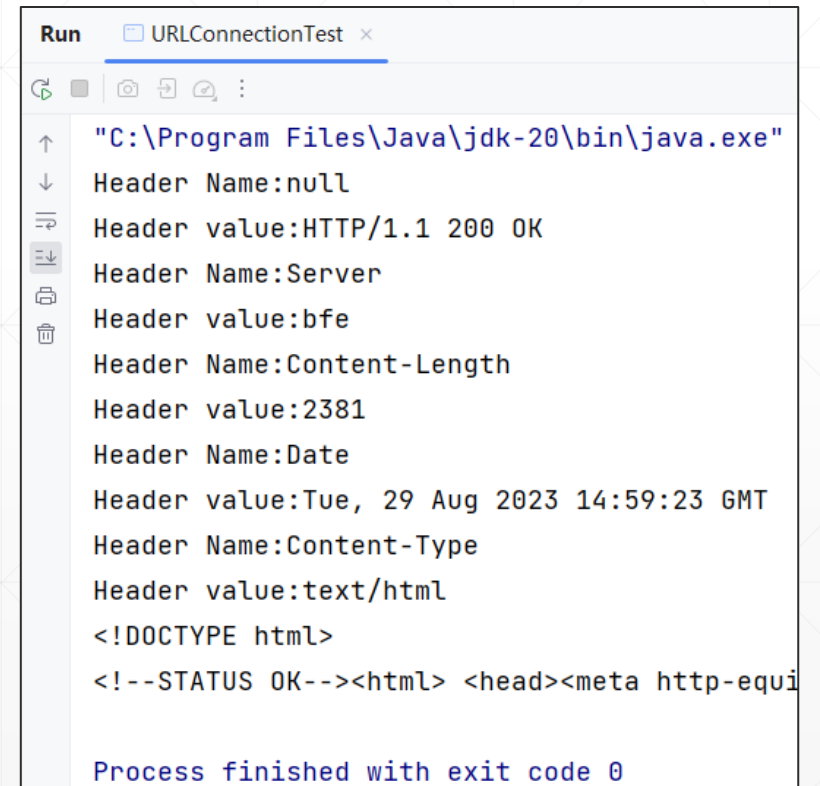
URLConnection对象也有一个getInputStream()方法用于获取输入流，同时，它还可以调用getHeaderFields()等方法获取服务端发回的HTTP响应信息。

使用URLConnection的示例代码

```
URL url = new URL("http://www.baidu.com/");
URLConnection urlConnection = url.openConnection();
//获取Server返回的HTTP响应首部
Map<String, List<String>> headers = urlConnection.getHeaderFields();
Set<Map.Entry<String, List<String>>> entrySet = headers.entrySet();
for (Map.Entry<String, List<String>> entry : entrySet) {
    String headerName = entry.getKey();
    System.out.println("Header Name:" + headerName);
    List<String> headerValues = entry.getValue();
    for (String value : headerValues) {
        System.out.print("Header value:" + value);
    }
    System.out.println();
}
//获取HTTP响应的Body
InputStream inputStream = urlConnection.getInputStream();
bufferedReader = new BufferedReader(
    new InputStreamReader(inputStream, StandardCharsets.UTF_8));
String line = bufferedReader.readLine();
while (line != null) {
    System.out.println(line);
    line = bufferedReader.readLine();
}
```

示例: URLConnectionTest.java

运行结果



```
Run  URLConnectionTest x
"C:\Program Files\Java\jdk-20\bin\java.exe"
Header Name:null
Header value:HTTP/1.1 200 OK
Header Name:Server
Header value:bfe
Header Name:Content-Length
Header value:2381
Header Name:Date
Header value:Tue, 29 Aug 2023 14:59:23 GMT
Header Name:Content-Type
Header value:text/html
<!DOCTYPE html>
<!--STATUS OK--><html> <head><meta http-equiv=
```

Process finished with exit code 0